Radio and Protocol Deep Dive — From Our C Code Down to the Silicon

All page references cite the **Semtech SX1261/2 Datasheet Rev 2.2, Dec 2024** (the 118-page PDF in your possession, `DS_SX1261_2 V2-2.pdf`). Where our own firmware is quoted, it is quoted verbatim from the repository; where Meshtastic's firmware is quoted, it is from `meshtastic/firmware` on GitHub.

---

0. The three links, precisely classified

The system has three wireless links, and they are three *different kinds* of link. Being precise about this is the foundation of everything below.

Link	Kind	Direction	Radio tech	Who implements it
Satellites → Heltec	broadcast reception	one-way, down only	GPS L1, 1575.42 MHz	the GNSS module (hardware)
Heltec ↔ Waveshare	connection-oriented client/server	two-way	Bluetooth Low Energy, 2.4 GHz	Meshtastic (server) + our `ble_link.cpp` (client)
Heltec ↔ Heltec	connectionless broadcast	two-way, symmetric	LoRa, 915 MHz ISM	Meshtastic (host MCU) + SX1262 (transceiver)

We never "communicate to the satellite." GPS is receive-only: thirty-odd satellites broadcast their position and an atomic timestamp continuously; the GNSS module hears at least four of them, solves the geometry, and emits the result as NMEA text sentences over UART at 1 Hz. There is no uplink, no handshake, no acknowledgment — which is why satellite count and sky view are the only things that matter for that link.

The division of labor on the LoRa link matters for everything below: the **ESP32-S3 running Meshtastic is the host MCU**, and the **SX1262 is a peripheral transceiver** it drives over a 4-wire SPI bus plus a BUSY handshake line and an interrupt line. The radio chip modulates, demodulates, and buffers one packet at a time; every decision about *what* to send and *when* lives in Meshtastic on the ESP32.

---

1. FSK: where it fits in this project (short answer: it doesn't — and why is the interesting part)

The SX1262 contains **two complete modems** sharing one RF front end (datasheet §6, p. 38): a **LoRa modem** and a **(G)FSK modem**, each with its own packet engine and frame format. The host selects one with the `SetPacketType(...)` command — the first command issued before any frequency, modulation, or packet configuration (p. 38). Meshtastic calls it with `PACKET_TYPE_LORA`. **The FSK modem is present in our silicon and used nowhere in our project.**

Why we can defend that choice, not just state it:

- **FSK encodes bits as frequency shifts** and needs the signal to be *stronger than the noise* to demodulate (positive SNR). It exists on this chip for legacy protocols and high-data-rate use (0.6–500 kb/s, §6.2.1 p. 43).
- **LoRa (chirp spread spectrum) demodulates below the noise floor** — at our SF11 the demodulator works down to **SNR = −17.5 dB** (Table 6-1, p. 39). That sub-noise margin *is* our kilometers of range.
- A revealing datasheet detail (p. 38): the LoRa modem rejects a co-channel GMSK/FSK interferer up to **19 dB stronger than itself**. LoRa and FSK users can share a band and barely notice each other — spread-spectrum orthogonality in action.

And Long Range FHSS (§6.3, pp. 49–50)? The chip supports LR-FHSS — a frequency-hopping GMSK mode that nearly matches SF12's link budget while letting thousands of devices share spectrum (it is used for direct-to- satellite LoRaWAN uplinks). But note the datasheet's fine print: the chip provides *only the modulation and autonomous hop timing*; frame construction, FEC, whitening, interleaving, and hop-sequence computation are "under the responsibility of the host MCU" (p. 50). Meshtastic implements none of that, so LR-FHSS is also unused here. If anyone asks "could this system talk to satellites?" — that is the honest door it would go through, and it is a different project.

---

2. Setting the US frequency range — the actual code path, all four layers

Layer 1 — our provisioning (what we type; `hardware/README.md`):

```
python -m meshtastic --port COMx --set lora.region US
```

(and when the CLI's port-close reset ate that write, the Python API with the port held open — the recovery is in the repo history.)

Layer 2 — Meshtastic's region table. In `meshtastic/firmware/src/mesh/RadioInterface.cpp` every legal region is one `RDEF` entry; ours is:

```
#define RDEF(name, freq_start, freq_end, duty_cycle, spacing, power_limit,
             audio_permitted, frequency_switching, wide_lora) ...

RDEF(US, 902.0f, 928.0f, 100, 0, 30, true, false, false),
```

That single line encodes the regulatory envelope: **902–928 MHz** (the FCC §15.247 ISM band), **100% duty cycle allowed** (unlike Europe's 10%/1% bands), **+30 dBm power ceiling**. Region UNSET has no envelope — which is why an unset radio refuses to transmit at all (we hit exactly this after the recovery re-flash).

Layer 3 — picking the channel inside the band. Also in `RadioInterface.cpp` (`applyModemConfig()`):

```
float freq = myRegion->freqStart + (bw / 2000) + (channel_num * (bw / 1000));
```

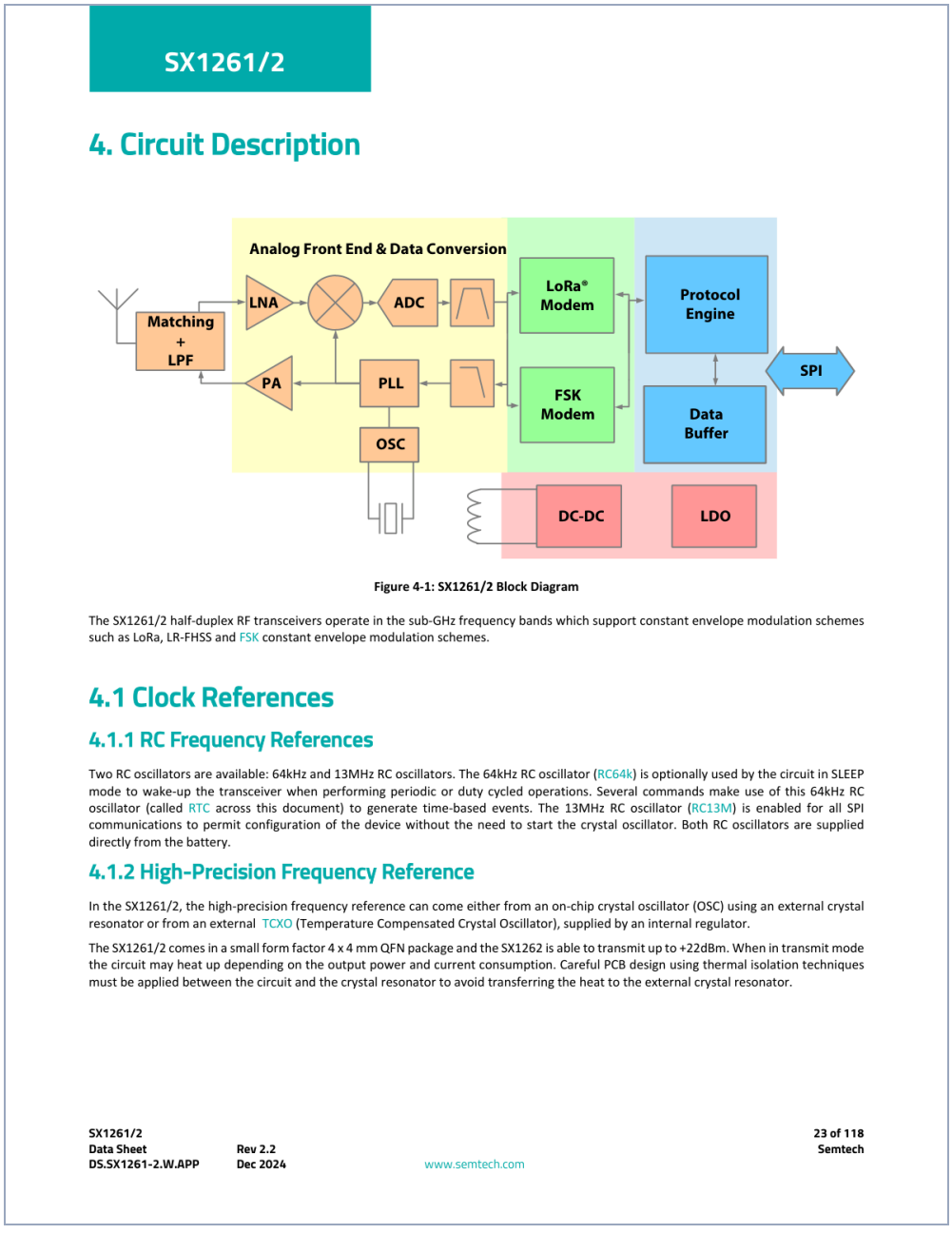
The 26 MHz band is sliced into bandwidth-wide slots; `channel_num` comes from a **hash of the channel name** ("Group9") modulo the number of slots. That is a quietly elegant trick: both radios compute the same hash from

the same channel name, so they land on the same frequency *with no coordination message ever sent* — the channel name in the shared URL is simultaneously the encryption domain and the frequency agreement.

Layer 4 — the silicon. Meshtastic's driver converts that float to the chip's `SetRfFrequency(...)` command; inside, the PLL synthesizes the carrier by fractional division of the 32 MHz crystal (§4.2, p. 26), and FSK or chirps are generated by modulating that divider ratio. This is the `PLL → PA → matching + LPF → antenna` path across the bottom of the block diagram on p. 23.

---

3. A guided tour of the SX1262 (block diagram, datasheet p. 23)



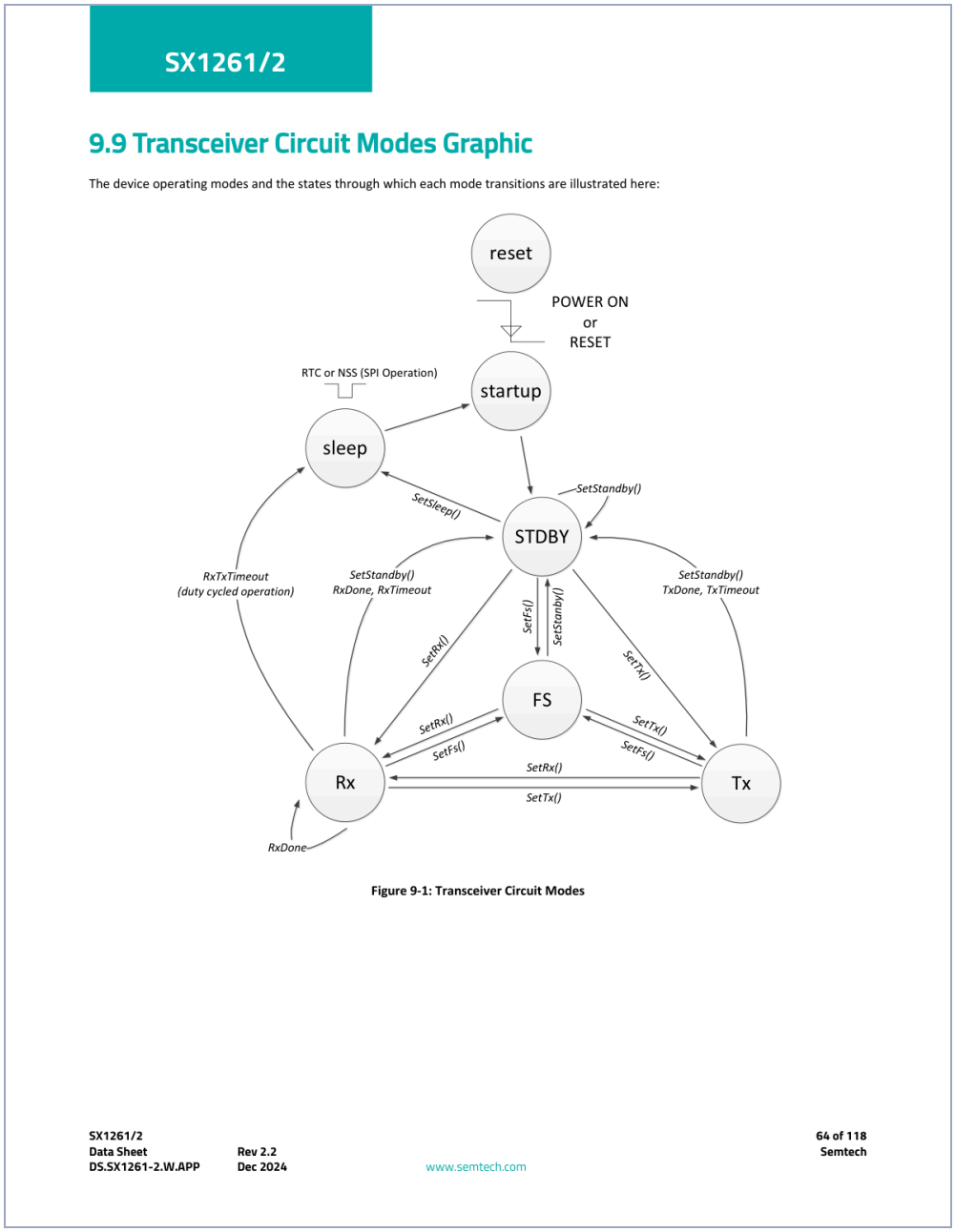
Datasheet p. 23, Figure 4-1: SX1261/2 block diagram

Walk Figure 4-1 left to right, receive path first:

- **Matching + LPF → LNA:** the antenna's captured signal — remember, this can be 15 dB *below* the noise — is impedance-matched, low-pass filtered, and amplified by the low-noise amplifier.
- **Mixer + PLL:** the RF signal is mixed down. A subtlety from p. 39: for bandwidths up to 250 kHz (ours), the chip does a **double conversion** — analog down to a low intermediate frequency, then digitally to baseband — which improves image rejection; only the 500 kHz mode goes straight to zero-IF.
- **ADC → digital modems:** from here on, everything is math. The **LoRa modem** correlates the incoming samples against the known chirp patterns (this correlation is the processing gain); the **FSK modem** sits beside it, powered but idle in our use.
- **Protocol engine + data buffer:** the packet engine handles preamble detection, header decode, de-interleaving, FEC, and CRC, writing payload bytes into the **256-byte RAM buffer** (§7, p. 51).
- **SPI:** the ESP32 reads that buffer out. On the transmit side the same path runs in reverse: buffer → packet engine → modem → PLL modulation → **PA** (the SX1262's +22 dBm amplifier, §4.4.2 p. 28) → filter → antenna.
- **DC-DC / LDO** (bottom): the power block — the reason the whole receive chain runs on ~4.2 mA, which is what makes battery soak tests boring, in the good sense.

---

4. The operating-mode state machine (datasheet §9.9, p. 64 — the circle diagram)



Datasheet p. 64, Figure 9-1: transceiver circuit modes

The transceiver is always in exactly one mode: **SLEEP, STDBY (RC or XOSC), FS, RX, TX** (§9, p. 60), plus CAD as a special receive activity. The circle diagram on p. 64 is the map; here is one Meshtastic position broadcast traced onto it:

1. **Idle = RX.** Meshtastic parks the radio in continuous receive — the mesh must always be listening. (BUSY goes low "as soon as the RX is up," §8.3.1 p. 55.)
2. **Decision to send** (smart broadcast fired): the ESP32 writes the ~40 protobuf bytes into the buffer with `WriteBuffer()`, sets `SetBufferBaseAddresses`, packet params, then —
3. **CAD first — listen before talk** (§6.1.5, p. 41). Below the noise floor, RSSI is useless for sensing the channel ("clearly impracticable," says the datasheet), so the chip has a dedicated **Channel Activity**

Detector: it runs the LoRa correlator for 1–16 symbols and raises an IRQ if it hears LoRa preamble *or* *data* symbols. This is the physical mechanism under Meshtastic's CSMA: if the channel is busy, back off (SNR-weighted), try again.

4. `SetTx()`: the chip passes through **FS mode** while the PLL locks onto our channel frequency, ramps the PA, and transmits: preamble (16 symbols in Meshtastic's config), explicit header at CR 4/8 with its own CRC, payload at CR 4/5, payload CRC (frame format §6.1.3, pp. 40–41). ~0.5 s later —
5. **TxDone IRQ** on the DIO1 pin wakes the ESP32; the chip auto-returns to STDBY, and Meshtastic immediately puts it back in **RX**.

Every one of those SPI exchanges is gated by the **BUSY line** (§8.3.1, p. 55): commands may only be sent when BUSY is low; after each write command the internal CPU raises BUSY (≤600 ns later) until the state transition completes. That little handshake is why the driver never has to guess whether the radio is ready.

---

5. The four LoRa knobs, with our values (datasheet §6.1.1, pp. 38–40)

The capability to receive signals with negative SNR increases the sensitivity, link budget and range of the LoRa receiver.

Table 6-1: Range of Spreading Factors (SF)

Spreading Factor (SF)[1]	5	6	7	8	9	10	11	12
2^SF (Chips / Symbol)	32	64	128	256	512	1024	2048	4096
Typical LoRa Demodulator SNR [dB]	-2.5	-5	-7.5	-10	-12.5	-15	-17.5	-20

1. Not all SF are available for any bandwidth with the SX1261/2

A higher spreading factor provides better receiver sensitivity at the expense of longer transmission times (time-on-air).

With a knowledge of the key parameters that can be selected by the user, the LoRa symbol rate is defined as:

$$Rs = \frac{BW}{2^{SF}}$$

where BW is the programmed bandwidth and SF is the spreading factor. The transmitted signal is a constant envelope signal. Equivalently, one chip is sent per second per Hz of bandwidth.

6.1.1.2 Bandwidth

An increase in signal bandwidth permits the use of a higher effective data rate, thus reducing transmission time at the expense of reduced sensitivity improvement.

The LoRa modem operates at a programmable bandwidth (BW_L) around a programmable central frequency f_{RF}

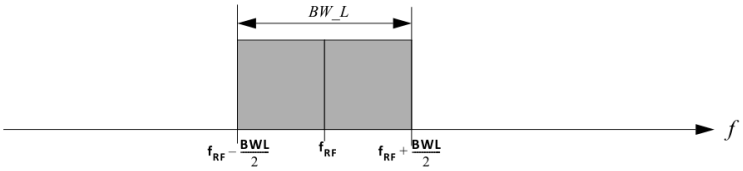


Figure 6-1: LoRa® Signal Bandwidth

An increase in LoRa signal bandwidth (BW_L) permits the use of a higher effective data rate, thus reducing transmission time at the expense of reduced sensitivity improvement. There are regulatory constraints in most countries on the permissible occupied bandwidth. The LoRa modem bandwidth always refers to the double side band (DSB). The range of LoRa signal bandwidths available is given in the table below:

Table 6-2: Signal Bandwidth Setting in LoRa® Mode

Signal Bandwidth	0	1	2	3	4	5	6	7	8	9
BW_L [kHz]	7.81	10.42	15.63	20.83	31.25	41.67	62.5	125	250 [1]	500 [1]

1. For RF frequencies below 400MHz, there is a scaling between the frequency and supported BW, some BW may not be available below 400MHz

For BW_L up to 250kHz, the receiver performs a double conversion. A first down conversion to low- IF is performed inside the RF chain, a second conversion to baseband is performed digitally inside the baseband modem. When the 500kHz bandwidth is used, a single down-conversion to zero-IF is performed in the RF part.

6.1.1.3 FEC Coding Rate

To further improve the robustness of the link the LoRa modem employs cyclic error coding to perform forward error detection and correction.

Datasheet p. 39: SF table (Table 6-1), bandwidths (Table 6-2), symbol-rate formula

`SetModulationParams(...)` takes exactly four parameters. Ours come from Meshtastic's LONG_FAST preset:

Knob	Ours	What it does	Datasheet
Spreading Factor	**SF11**	2^{11} = 2048 chips/symbol; sets the SNR floor at **−17.5 dB**	Table 6-1, p. 39
Bandwidth	**250 kHz**	chip rate = BW (one chip per second per Hz); symbol rate Rs = BW/2^SF	§6.1.1.2, p. 39
Coding Rate	**4/5**	FEC: every 4 data bits travel as 5; "best trade-off in normal conditions"	Table 6-3, p. 40

Knob	Ours	What it does	Datasheet
LDRO	**off**	see below	§6.1.1.4, p. 40

Symbol time, from the datasheet's own formula (p. 39): T_sym = 2^SF / BW = 2048 / 250,000 = **8.19 ms**. A ~40-byte position packet, with preamble, header, FEC, and CRC, needs roughly 60 symbols → **≈ 0.5 s time-on-air**. (The datasheet's §6.1.4, p. 41, defers the exact closed-form to Semtech's SX126x drivers and AN1200.22 — that is why our deck cites the app note for the airtime equation.)

Low Data Rate Optimization (p. 40) — one of the terms you asked about, and a satisfying case of the datasheet giving an exact rule: LDRO drops two bits per symbol to help the receiver track very slow symbols, and is "usually recommended when a LoRa symbol time is equal or above **16.38 ms**." Our symbol is 8.19 ms — exactly half the threshold — so **LDRO is off** for LONG_FAST. (Flip to SF11 at 125 kHz, or SF12, and it would switch on. Meshtastic's driver applies this rule automatically.)

RSSI calibration (§6.1.6, pp. 41–42) — why it exists: the RSSI the chip reports is skewed by everything between the antenna and the die (matching network, RF switch). Miscalibrated RSSI picks wrong front-end gains, causing missed packets. The chip ships calibrated for the 868/915 MHz reference design (Table 6-4), which the Heltec follows closely — but this section is the reason Victor's range logs should record RSSI *trends*, not treat absolute dBm as gospel truth.

---

6. The data buffer (§7, pp. 51–52) — where our 40 bytes physically live

One 256-byte RAM, shared by TX and RX, base addresses set by `SetBufferBaseAddresses()` (both default 0x00). On transmit, the packet engine reads `payloadLength` bytes from `TxBaseAddr` and streams them into the modem. On receive, payload bytes land at `RxBaseAddr`, and `GetRxBufferStatus()` tells the host where and how many.

Three details worth knowing:

- The buffer **survives every mode except sleep** (sleep clears it).
- **CRC-failed packets are still written** to the buffer — the host may inspect corrupted data (Meshtastic just drops them, but the option shapes the driver's IRQ handling).
- Our entire position packet uses ~40 of 256 bytes — one packet at a time. The chip is not a queue; **Meshtastic's queues live on the ESP32**, and the FromRadio queue our display drains lives yet another layer up. Three buffers at three layers, each sized to its job.

---

7. SPI, BUSY, and IRQ (§8, pp. 53–58) — how the ESP32 actually drives the chip

- **SPI:** mode 0 (CPOL=0, CPHA=0), up to **16 MHz**, command-based: an opcode byte plus parameters (§8.2, p. 53). Commands configure and control; `WriteBuffer`/`ReadBuffer` move payloads.
- **BUSY (mandatory):** the ready/not-ready handshake described in §4 above. Waking from sleep, the NSS falling edge itself triggers boot; the host must then wait for BUSY to fall before the first command (§8.2.2, p. 54).

- **DIO1 = interrupt line:** TxDone, RxDone, CAD done, timeouts — the chip signals events; the ESP32's driver ISR reads and clears them (§8.5, p. 58). This is event-driven at the silicon level, the same design philosophy as our FromNum "doorbell" one layer up — nothing in this stack polls when it can be interrupted.

On the Heltec, these lines (SPI ×4, BUSY, DIO1, NRESET) are the actual copper between the ESP32-S3 and the SX1262 — the "SPI" arrow on our hardware diagram, now with names.

---

8. Claim-to-page map (the citations you asked for)

Claim in our deck / docs	Where in the datasheet
SF11 SNR limit −17.5 dB ("15 dB below noise" after noise-figure math)	**Table 6-1, p. 39**; receiver specs §3.5.3, p. 20
2048 chips per symbol at SF11	Table 6-1, p. 39
Symbol time Rs = BW/2^SF → 8.19 ms	§6.1.1.1–6.1.1.2, p. 39
~0.5 s time-on-air per position	§6.1.4, p. 41 (defers to SX126x drivers + AN1200.22)
CAD as the carrier-sense for CSMA	§6.1.5, p. 41
LDRO on/off rule (16.38 ms symbol threshold)	§6.1.1.4, p. 40
Explicit header, CR 4/8 header FEC, own CRC	§6.1.3.1, p. 41
FSK modem exists, 0.6–500 kb/s, unused by us	§6.2, pp. 43–48
LR-FHSS modulation-only support (host does the modem work)	§6.3, pp. 49–50
256-byte data buffer, RX/TX pointers, sleep clears it	§7, pp. 51–52
SPI 16 MHz mode 0; BUSY handshake ≤600 ns; wake timing	§8.2–8.3, pp. 53–55
Operating modes circle diagram (reset→startup→STDBY→FS→TX/RX)	§9.9, p. 64 (modes detail §9.1–9.8, pp. 60–63)
+22 dBm PA (SX1262)	§4.4.2, p. 28; TX specs §3.5.4, p. 22
PLL fractional-N carrier synthesis	§4.2, p. 26
Block diagram (LNA→mixer→ADC→modems→engine→SPI)	Figure 4-1, p. 23

---

9. The whole stack, one packet, every layer named

```
[Partner vehicle]                                [Our vehicle]
GPS satellites ──L1 broadcast──▶ GNSS module
GNSS ──NMEA/UART 1 Hz──▶ ESP32 (Meshtastic)
Meshtastic: protobuf Position (~40 B)
ESP32 ──SPI: WriteBuffer + SetTx──▶ SX1262
SX1262: CAD ▸ FS (PLL lock) ▸ TX
  chirps @ SF11/250 kHz, CR 4/5, ~0.5 s
  ·····------------ 915 MHz, kilometers ------------·····
                                       SX1262: RX, correlator digs the
```

```
                                        signal out from below noise,
                                        FEC repairs, CRC checks,
                                        payload → 256 B buffer
                                      RxDone IRQ ──DIO1──▶ ESP32
                                      ESP32 ──SPI ReadBuffer──▶ decrypt
                                        (AES-256) ▸ node DB ▸ queue on
                                        FromRadio ▸ notify FromNum
                                      our ble_link drains the queue
                                      nanopb decodes ▸ node_table
                                      ▸ TrackerState ▸ dead reckoning
                                      ▸ nav math ▸ arrow on the RLCD
```

Everything above the dotted line on the receive side happens inside stock components; everything after "our ble_link" is the custom firmware. The protocol role — your role — is the hinge between the two worlds, which is why understanding both sides of the hinge is worth this document.